Neural Networks and Deep Learning

BMED365: Topic List D01–D13

BMED365

Spring 2027

Overview

1. Basic Neural Networks
2. Training Neural Networks
3. Convolutional Neural Networks (CNN)
4. Regularization and Advanced Techniques

D01: Compare biological and artificial neurons

Biological neuron:

- **Dendrites:** Receives signals
- **Cell body:** Processes
- **Axon:** Transmits
- **Synapse:** Connection to next neuron
- Complex electrochemical activity

Artificial neuron (perceptron):

- **Inputs** $x_1, x_2, \ldots$: Receives data
- **Weights** $w_1, w_2, \ldots$: Synaptic strength
- **Sum:** $z = \sum w_i x_i + b$
- **Activation:** $a = f(z)$ (f = activation function)
- Mathematical simplification

Key similarity

Both sum incoming signals and "fire" (activate) if total stimulation exceeds a threshold.

Important difference

Artificial neurons are **greatly simplified models** - they do not capture the full complexity of the brain.

D02: Describe the structure of a multilayer perceptron (MLP)

MLP architecture:

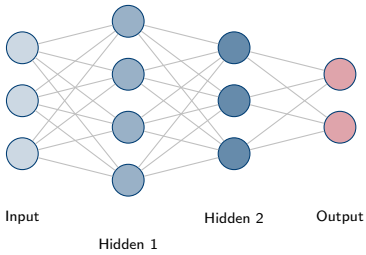
- **Input layer:** Receives features (no processing)
- **Hidden layers:** One or more layers of neurons
- **Output layer:** Produces prediction

Fully connected (dense):

- Each neuron in one layer is connected to **all** neurons in the next layer
- Many parameters (weights and biases)

"Deep" learning:

- ≥ 2 hidden layers = deep model
- More layers $\rightarrow$ more abstract representations



D03: Explain what an activation function is (ReLU, sigmoid)

Why activation functions? Without them: The entire network = linear transformation. They introduce **non-linearity**.

Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$

- Output: $(0, 1)$
- Problem: *vanishing gradients*
- Used in output for binary classification

ReLU: $\text{ReLU}(z) = \max(0, z)$

- Output: $[0, \infty)$
- Modern standard for hidden layers
- Fast, avoids vanishing gradients

Softmax (multi-class output)

$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$ - gives probability distribution over classes (sums to 1)

D04: Understand the concept of forward propagation

Forward propagation = forward computation

1. **Input:** Data x (features) is fed into the input layer
2. **For each layer:**
   - Compute weighted sum: $z^{(l)} = W^{(l)} \cdot a^{(l-1)} + b^{(l)}$
   - Apply activation: $a^{(l)} = f(z^{(l)})$
3. **Output:** Prediction $\hat{y}$ from the final layer



Key point

Forward propagation calculates prediction - no learning happens here. Learning happens through backpropagation and weight updates.

D05: Explain backpropagation at a conceptual level

Backpropagation = backward computation of error

Main idea:

1. **Compute error:** Compare prediction $\hat{y}$ with ground truth y
2. **Propagate backward:** How much did each weight contribute to the error?
3. **Chain rule:** Differentiate error with respect to each weight in the network
4. **Update weights:** Adjust to reduce error

Intuition

Imagine adjusting a complex machine: Backpropagation tells you which screws (weights) to turn, and in which direction, to improve the result.

Mathematical (simplified)

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

(Chain rule for derivatives propagates gradients backward through the network)

D06: Understand gradient descent and learning rate

Gradient descent:

- Optimization algorithm to **minimize the loss function**
- Follows the **negative gradient** (steepest descent)

Update rule:

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \frac{\partial L}{\partial w}$$

Learning rate η:

- Determines **step size**
- Too large: Jumps over minimum
- Too small: Slow convergence
- Typical: 0.001–0.01

Variants:

- **Batch GD:** All data per update
- **Stochastic GD:** One example at a time
- **Mini-batch GD:** Small batches (32–128)
- **Adam:** Adaptive learning rate (popular!)

Epoch

An **epoch** = one complete pass through the entire training dataset

D07: Know about loss functions (cross-entropy, MSE)

Loss function:

- Measures **how wrong** the model's predictions are
- The goal: Minimize loss during training

MSE (Mean Squared Error):

$$L = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- Used for **regression**
- Penalizes large errors heavily
- Continuous output

Cross-Entropy:

$$L = -\sum_{i} y_i \log(\hat{y}_i)$$

- Used for **classification**
- Binary: $-[y \log \hat{y} + (1-y)\log(1-\hat{y})]$
- Measures divergence between probability distributions

Choosing the right loss

Regression: MSE/MAE **Binary:** Binary CE **Multi-class:** Categorical CE

D08: Describe a convolutional neural network (CNN)

CNN = specialized for image data

- Exploits **spatial structure** in images
- Far fewer parameters than fully connected network
- Translation invariant - recognizes patterns regardless of position

Typical CNN architecture:



Hierarchical learning:

- Early layers: Simple features (edges, textures)
- Later layers: Complex features (shapes, object parts)
- Final layer: High-level concepts (objects, categories)

D09: Explain what a convolution filter does

Convolution filter (kernel):

- Small matrix (e.g. 3×3) that **slides over the image**
- Calculates dot product between filter and image patch
- Produces a **feature map**

Example: Edge detection

$$\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

- Detects horizontal edges
- Different filters $\rightarrow$ different features
- CNN **learns** optimal filters!

Key concepts:

- **Stride:** How far the filter moves
- **Padding:** Adding pixels at the border
- **Channels:** RGB = 3 channels
- **Feature map:** Output from convolution

Weight sharing

Same filter used across entire image $\rightarrow$ dramatically fewer parameters than a fully connected network.

D10: Describe pooling layers and their function

Pooling = downsampling of feature maps

Most common type: Max pooling

- Selects **maximum value** in each region (e.g. 2×2)
- Reduces spatial size by factor of 2

Advantages of pooling:

- **Reduces computation** - fewer parameters
- **Translation invariance** - slight shift does not affect output
- **Prevents overfitting**

Example (2×2 max pool):

$$\begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix} \rightarrow 4$$

$$\begin{bmatrix} 5 & 1 \\ 0 & 2 \end{bmatrix} \rightarrow 5$$

Other pooling types

- **Average:** Mean value
- **Global avg:** Feature map $\rightarrow$ one value

D11: Know about batch normalization and dropout

Batch Normalization:

- Normalizes activations in each mini-batch
- $\hat{x} = \frac{x-\mu}{\sigma}$, then scale/shift
- Advantages:
  - Faster training
  - Stabilizes learning
  - Allows higher learning rate
- Typically placed after convolution, before activation

Dropout:

- "Turns off" random neurons during training
- Typical dropout rate: 0.2–0.5
- Advantages:
  - Reduces **overfitting**
  - Forces network to be robust
  - Acts as ensemble
- Used only during training!

Regularization techniques

Both batch normalization and dropout help prevent overfitting and improve generalization.

D12: Understand the concept of transfer learning

Transfer learning:

- Reuse a model trained on one problem for a new problem
- Especially useful when you have **limited data**

Typical approach:

1. Take a pretrained model (e.g. trained on ImageNet - millions of images)
2. Freeze early layers (general features)
3. Replace/train final layer for your specific task
4. (Optional) Fine-tune entire model with low learning rate

Why does it work?

Early layers learn **general features** (edges, textures) that are useful for many tasks. Only the final layers are task-specific.

Example in medicine

Train on millions of natural images → Fine-tune on thousands of X-ray images → Better results than training from scratch!

D13: Know about advanced architectures (ResNet, ViT)

ResNet (Residual Networks):

- Introduced **skip connections**
- Allows gradients to flow directly
- Enables very deep networks (50–152+ layers)
- $y = F(x) + x$ (residual block)

Important contribution:

- Solved the problem of degradation in deep networks
- Standard for medical image analysis

ViT (Vision Transformer):

- Applies transformer architecture to images
- Divides image into patches ("tokens")
- Uses self-attention
- Scales well with data and compute

Modern trend:

- Outperforms CNN on large datasets
- Used in state-of-the-art models

Other important architectures

U-Net: Segmentation (medical classic) | **EfficientNet**: Balanced scaling | **DenseNet**: Dense connections

Summary: Deep Learning

Fundamentals:

- D01–D03: Neurons, MLP, activation functions
- D04–D07: Forward/backprop, gradient descent, loss functions

CNN:

- D08–D10: Architecture, convolution filters, pooling
- Hierarchical learning of image features

Advanced:

- D11: Batch normalization, dropout (regularization)
- D12: Transfer learning (reuse of pretrained knowledge)
- D13: ResNet (deep networks), ViT (transformers for images)

Lab 2 - Hands-on experience

Build, train and evaluate CNNs with PyTorch. Use Grad-CAM to understand model decisions.